
One Operator, Larger Graphs: Learning to Compare Symbolic Expressions

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Can rewriting mathematics with a single operation make it easier for a neural
2 model to learn? We study expressions built from $E(a, b) = \exp(a) - \log b$, an
3 operation that can express a broad range of elementary functions. A uniform
4 vocabulary comes at a structural cost: under the compiler studied here, expression
5 trees become about 41 times larger at the median. Sharing repeated expressions
6 reduces this cost, but compiled graphs still contain about ten times as many nodes
7 on average. We then ask whether graph models can learn to recognize equivalent
8 expressions. Small controlled experiments separate missing input information
9 from unsuccessful training. Giving nodes information about their position in
10 the whole graph allows models that previously failed to fit to learn their training
11 examples. In the main comparison, all nine final models fit their training sets, yet
12 their mean accuracy remains near chance on a test that swaps variable roles while
13 matching measured structural features. A separate compiler for bounded rational
14 polynomials ensures that the construction stays within the real-valued domains
15 of its primitive operations; numerical checks provide additional, finite evidence
16 about its implementation. Together, these results show why representation size,
17 successful training, and generalization must be assessed separately when designing
18 learned components for symbolic reasoning. Neural equivalence scores remain
19 suggestions for an independent checker, not proofs.

20 1 Introduction

21 The expressions $x + y$ and $y + x$ look different but compute the same function. A learned equivalence
22 model tries to recognize such relationships from examples. Its job depends on how the expressions
23 are represented: a familiar operation such as multiplication might be one node in an expression graph
24 or a long sequence of more basic operations.

25 Odrzywołek [2026] shows that a single operation,

$$E(a, b) = \exp(a) - \log b, \quad (1)$$

26 together with the constant one, can express a scientific-calculator basis. We call this representation
27 EML. Expressive sufficiency does not tell us how large a compiled expression becomes, whether its
28 intermediate calculations remain in the intended domain, or how easily a learned model can use it.
29 Those are the questions studied here.

30 They matter for verifiable coding because learned components can rank proposed rewrites, order
31 equivalence candidates, or guide proof search. A small primitive vocabulary is attractive in such
32 systems, but it can move useful information into long graph paths. High accuracy on generated
33 examples can also conceal failures on simple changes in variable or coefficient binding. We do not
34 evaluate a proof assistant or a code-generation system. We study a representation choice that such

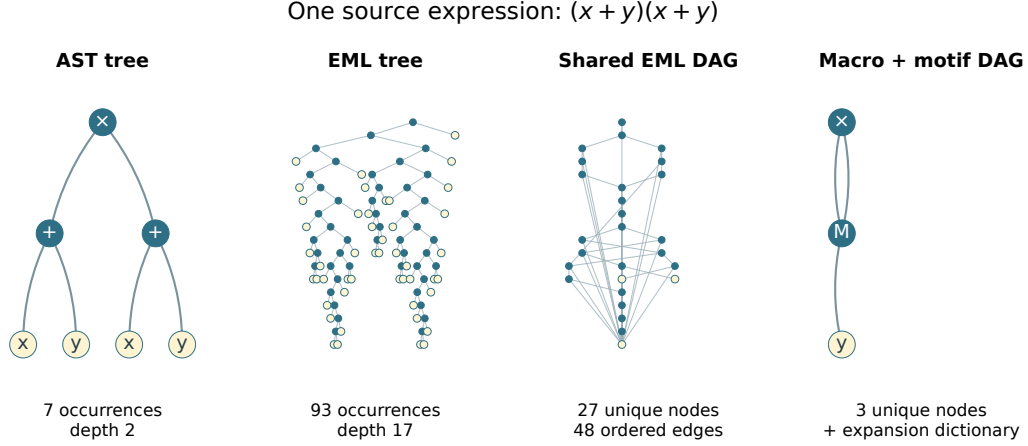


Figure 1: A small expression can become a much larger compiled graph. For $(x + y)(x + y)$, the source tree has seven occurrences and its shared graph has four nodes. Historical EML has 93 tree occurrences and 27 shared nodes. A three-node motif view hides repeated patterns behind a dictionary, whose definitions must also be counted. This example uses the historical compiler.

35 systems could make; a sound pipeline would still require an independent symbolic checker to accept
36 each proposed step.

37 Our contributions are three results: structural costs of one EML compiler on 250,000 expressions;
38 controlled experiments separating available information, fitting, and generalization; and a real-valued
39 construction for bounded rational polynomials, evaluated alongside source and historical EML graphs.

40 Learning semantic representations of symbolic expressions predates this work [Allamanis et al.,
41 2017]; neural symbolic mathematics also has a substantial literature [Lample and Charton, 2020,
42 Saxton et al., 2019]. Our contribution is the EML-specific cost and learning evidence, rather than the
43 idea of learned equivalence. Likewise, local graph-network limits and the use of graph positions or
44 global communication are established ideas [Xu et al., 2019, 2018, Rampasek et al., 2022]. We use
45 them as diagnostic tools and interventions, not as new general theories.

46 2 Representations and study design

47 An expression tree has one node for each operation, variable, or constant occurrence. A shared
48 expression graph stores repeated subexpressions once. Such a graph is a directed acyclic graph
49 (DAG): following its dependencies never returns to the starting node. We compare three inputs:

- 50 • **Source expression graph (AST).** Derived from an abstract syntax tree, it keeps familiar
51 operations such as addition and multiplication explicit and shares repeated subexpressions.
- 52 • **Historical EML.** The fixed `official_v4` compiler rewrites these operations into E and
53 shares repeated terms.
- 54 • **Bounded EML.** A separate compiler handles exact rational polynomials over declared
55 rational variable bounds. Its real-valued construction is described below. It changes graph
56 structure as well as domain behavior.

57 Exact polynomial calculation supplies the labels in the controlled learning study: equivalent pairs
58 have identical rational coefficients, while each unequal pair has an explicit assignment where its
59 values differ. This verifies the source labels. Checking a compiled graph is a separate task. Numerical
60 agreement at tested assignments does not prove agreement everywhere.

61 **A real-valued construction on a restricted domain.** The bounded compiler accepts exact integer
62 and rational constants, variables, addition, and multiplication. Fixed nonnegative integer powers
63 are written as repeated multiplication. The experiments use six variables, each between $1/4$ and 4 ;

Table 1: Which experiment supports which claim. The large structural population and the smaller learning populations are separate. “Selected” means the saved model with the lowest validation loss.

Population	Inputs and model	Model choice / semantics	Supported claim
250,000 expressions	Source and historical EML; no learner	Fixed compiler; no semantic guarantee needed for counting	Structural expansion
Historical learning: 50,000 training pairs	Source and historical EML; three local layers	Validation selection; compiled semantics partly unresolved	Generated-task performance under that setup
32-pair diagnostics	Source and historical EML; local and positional models	Training-only controls; old EML domain limits remain	Architecture-dependent fitting and measured signal size
480 training, 120 validation; three seeds	All three graphs; twelve positional layers	Selected models primary; final models diagnostic; new bounded construction separate	Fitting, model selection, and performance on two 600-pair tests

the construction extends to other declared rational bounds. It does not cover transcendental source expressions.

The main idea is to keep every logarithm’s argument positive. For example, $E(t, 1) = \exp(t)$, and a short composition gives $\log(t)$ when $t > 0$. Exact interval bounds tell the compiler when to add a positive offset before using a subtraction identity and remove the offset afterward. Products that cross zero can similarly be rewritten using positive shifted factors. Induction over the source expression gives value preservation and valid real-valued primitive domains, using the stated identities and exact interval bounds. Appendix B gives the construction.

All 21,107 retained polynomial source expressions compile. Direct evaluation of their primitive graphs gives 126,642 agreements: 21,107 graphs at three assignments and two numerical precisions. Each whole-graph evaluation checks every visited primitive operation. This is not a count of expressions, primitive nodes, or proof certificates. Nor is 21,107 a measured coverage fraction of the 250,000-expression corpus; coverage under this restricted grammar was not measured there. Finite real values can still be extremely large or poorly conditioned for floating-point computation. The new construction does not repair the previously trained historical graphs.

LLM assistance. OpenAI Codex assisted with coding, debugging, experimental design and execution, analysis scripts, interpretation of results, and manuscript refinement. Labels, compiler checks, and metrics were computed by executable symbolic or numerical procedures and model runs, rather than accepting LLM judgments as ground truth. AI-Scientist reviews run through Codex provided author-side feedback. LLMs were not components of the evaluated graph models or the exact checker at inference time. The authors remain responsible for the methods, results, and manuscript.

3 How much larger are the graphs?

The large structural population. The 250,000-expression population was generated with seed 20260721 and fixed family quotas: 70,000 algebraic expressions; 40,000 involving powers, division, or rational functions; 40,000 exponential/logarithmic; 40,000 trigonometric/hyperbolic; 35,000 mixed elementary; and 25,000 distribution-stress expressions. Generation recorded 286,413 attempts, 35,768 duplicate rejections, and 645 trivial-expression rejections. The saved split contains 175,000 training, 25,000 validation, 25,000 ordinary-test, and 25,000 distribution-stress expressions. These names describe this historical corpus, not the later 480-pair learning study.

The quotas define a designed benchmark distribution. They are not estimates of how often these operations occur in practical mathematics or code. The supplement contains the generation and analysis source and summary records, but excludes the full historical corpus. Consequently, it does not provide a self-contained replay of all 250,000 original measurements. Results broken down by source size and operation family would better characterize the large expansion tail; the aggregate figures alone do not supply that explanation.

99 The sampling grammar contains exact integers from -9 to 9 , rationals with numerators in
100 that range and denominators $2-9$, six variables, arithmetic operations, powers with exponents
101 $-3, -2, -1, 2, 3, 4$, exponentials, logarithms, and six trigonometric/hyperbolic operations. Domain
102 restrictions are enforced during generation. Ordinary target-size ranges $2-8, 9-16, 17-32, 33-64$,
103 and $65-128$ receive probabilities $.28, .27, .25, .15$, and $.05$; the stress profile uses the last three ranges
104 with probabilities $.15, .35$, and $.50$. Target depths reach 12 . After lowering to binary trees, actual
105 source sizes have median 29 , 95th percentile 123 , and range $2-213$; actual depths range from 1 to 17 .
106 These target and realized distributions differ because domain-preserving construction and lowering
107 can add nodes. The supplement supplies the full per-family operator, depth, variable-count, rejection,
108 and split configuration plus deterministic generation commands. It identifies the production source
109 and compiler versions; we have not rerun the full corpus for this analysis.

110 On the 250,000-expression corpus, the historical compiler increases tree size by a median of $40.66\times$
111 and a mean of $952.14\times$. The difference reflects a long tail of large expansions. Sharing repeated
112 expressions reduces the cost, but the mean per-expression ratio of compiled to source graph size
113 is still 10.47 . These are empirical measurements of a fixed compiler, not lower bounds for every
114 possible EML encoding.

115 Dictionary-based compression can shorten the visible graph further, but a dictionary is part of the
116 representation and has its own cost [Rissanen, 1978]. Figure 1 illustrates that distinction. The primary
117 learning comparison uses plain graphs and therefore avoids the motif dictionaries’ training-exposure
118 complication.

Table 2: Paired graph costs on the same 2,700 source expressions in the current development and two test populations. Depth is the longest dependency path. The ratio averages per-expression node ratios to the source graph.

Input	Nodes median	Nodes p95	Depth median	Depth p95	Mean ratio
Source	12	19	3	6	1.00
Historical EML	104	184	43	69	10.1
Bounded EML	249	792	103	183	28.9

119 Changing from historical to bounded EML also changes node counts and distances. Any learning
120 difference between those two variants therefore combines several changes; it cannot be interpreted as
121 the isolated effect of semantic validity.

122 4 What allows the models to fit?

123 The local graph model repeatedly combines each node’s features with information from its neighbors.
124 With a small number of rounds, information cannot travel far. A standard color-refinement test [Xu
125 et al., 2019] checks an upper bound on what this form of local processing can distinguish: it repeatedly
126 replaces each node’s label with its current label and its labeled neighbors. Identical resulting graph
127 summaries imply an obstruction for the specified model class. Different summaries only permit
128 separation; they do not show that training will find it.

129 The earlier three-layer EML model encounters this obstruction on matched examples. Increasing
130 to twelve layers removes that measured obstruction but does not, by itself, produce a successful fit.
131 On 32 balanced development pairs, we apply six training and graph-summary choices equally to
132 source and EML models. Five source-graph choices pass the training criterion; none of the six EML
133 choices does. Here and below, a fitting pass requires training accuracy at least 95% and prediction
134 loss (binary cross entropy, BCE) at most 0.2. A balanced constant prediction has loss approximately
135 0.693. The tiny controls require three consecutive passing checks.

136 **Small signals are a measurement, not a complete explanation.** We measure how far apart the
137 local model places matched graph summaries, relative to their overall magnitude. With a fixed
138 64-number summary and each model’s saved weights evaluated in 64-bit arithmetic (FP64), the
139 initial median relative difference is 4.8×10^{-3} for source graphs and 4.1×10^{-11} for historical
140 EML. This comparison is unchanged if all summaries are multiplied by a common constant. In
141 standard 32-bit arithmetic (FP32), changing the order of messages can cause changes comparable to

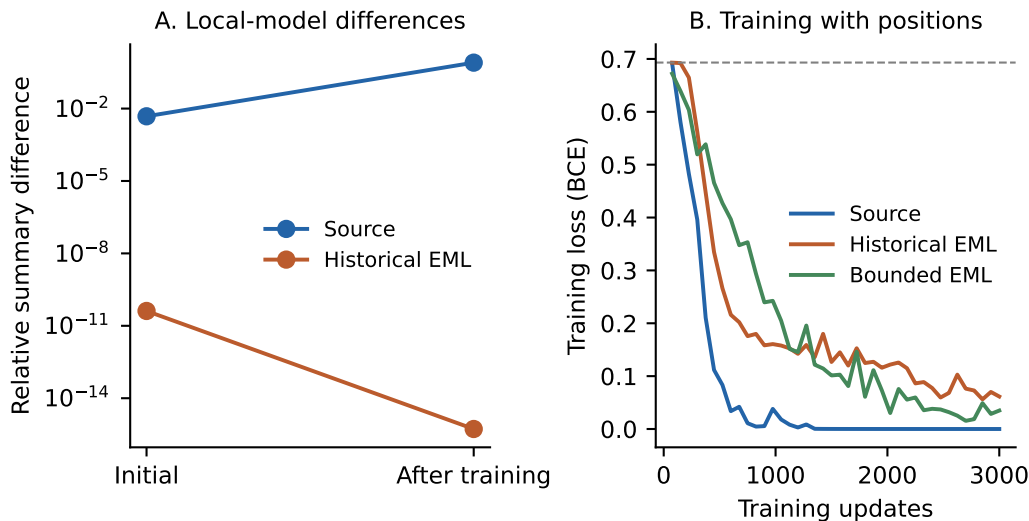


Figure 2: Two different pieces of evidence. Left: matched-summary distance divided by mean summary norm, using identical local-model weights in FP64. Right: training loss with global position information, for seed 20260921; the dashed line is constant-prediction loss. The plots concern different architectures and do not identify one unique cause of failed training. All main-study training seeds and fitting criteria are in Table 5.

the measured distinguishing signal. Higher-precision training still fails to fit the EML controls. These measurements show very small signals, but precision alone is not an adequate explanation or remedy.

Positions make a useful difference. We tell each node its shortest distance from the output and its longest distance from an input leaf, capping each value at 63. Computing these positions uses the whole graph before local processing begins. All sixteen tiny controls spanning source/EML inputs, three/twelve rounds, two seeds, and shared/separate layer weights pass. This demonstrates fitting on the tested examples. The intervention changes both information and optimization, so its success does not identify a unique mechanism.

The model averages node summaries within each layer and then averages across layers, always producing 64 numbers per graph. Its comparison network receives the sum, absolute difference, and elementwise product of the two summaries: 192 numbers at either depth. The shared-weight control repeats exactly the same block and has the same parameter count at three and twelve rounds. This addresses specific capacity differences; it does not turn the older concatenated-readout comparison into an isolated depth experiment.

A separate variable-binding control. On the 300 variable-swap test groups, the full-feature refinement check gives source: 300/300 distinguishable; historical eml: 300/300 distinguishable; bounded eml: 300/300 distinguishable. The check includes variable labels, numerical features, root flags, ordered edges, and the actual clipped positions. On 32 new, disjoint training pairs made by the same variable-swap construction, the unchanged positional recipe passes in 3/3 source, 0/3 historical-EML, and 3/3 bounded-EML runs. A separate saved-input check also distinguishes every training and validation group in this small control for each representation. The earlier successful position controls use different unequal examples; their fitting result does not carry over automatically to variable swaps. Distinguishable inputs rule out that measured obstruction, but do not explain why a particular training run fails. Complete outcomes, including unresolved checks and failed fits, are retained in the appendix.

Table 3 also shows why graph size alone is not an explanation of fitting. The bounded graphs are larger than the historical graphs in the main population, yet their models fit all three small variable-swap controls while the historical models fit none. The graph transformations differ in their paths, repeated subexpressions, and distribution of labels. The controls establish that success depends on

Table 3: Completed fitting controls. Each entry counts passes among all tested configurations or runs; a dash means the representation was not part of that study. Every pass requires both training accuracy $\geq 95\%$ and $\text{BCE} \leq 0.2$. The first two rows use the earlier small development task; the third uses new variable-swap negatives. Their different examples prevent treating the rows as one controlled ablation.

Control	Source	Historical EML	Bounded EML
Six training and graph-summary choices, without whole-graph positions	5/6	0/6	–
Positions; 3/12 rounds, shared/separate weights, two seeds	8/8	8/8	–
Positions; 32 variable-swap pairs, three seeds	3/3	0/3	3/3

the representation, example construction, and learner together; they do not isolate which of those structural differences causes historical EML’s failure.

The shared-weight positional comparison is a limited architecture control: changing the number of message-passing rounds leaves its learned parameter count unchanged. Both depths fit the tiny task. It is not a comparison of unrelated model families, and the main transfer study still uses one family of message-passing models. In particular, independently summarizing the two expressions may discard relationships that a model comparing their nodes directly could use. No completed experiment here measures the effect of that alternative.

The position cap affects the representations differently. The common cap of 63 clips leaf-height features for 37.9% of bounded-EML nodes, 2.0% of historical-EML nodes, and no source nodes; it also clips root-distance features for 14.0% of bounded-EML nodes. Equal caps therefore give unequal positional resolution. All complete graph inputs in the refinement check remain distinguishable, but that does not show that the lost position detail is harmless for learning. The fixed-recipe comparison remains valid within this constraint. A cap comparison with equal embedding capacity and a model with direct cross-expression communication has not been completed.

5 Fitting and generalization are different outcomes

The main controlled study uses the same 480 training pairs and 120 validation pairs for all three graph representations, with three random seeds. Training uses commutativity, distributivity, and regrouping. Testing also includes binomial squares, differences of squares, and cubic expansion, with familiar and unfamiliar numerical literals. All operators are present during training. These are six generator families, not a broad sample of mathematics.

The common model has twelve layers of width 64 and 457,741 parameters for each representation. Both source and historical EML models passed the tiny development checks before this recipe was selected. Training uses FP32 and AdamW at learning rate 10^{-3} , with a common limit of 3,000 updates, 200 epochs, or 1,800 active seconds. Validation loss selects a saved model every five epochs. Appendix A gives the complete specification.

Two tests ask different questions. Each test contains 600 pairs: 25 matched positive/negative groups in each of six template families and two literal groups. The tests share no source expressions, polynomial functions, or generator contexts with each other or the earlier populations. A context is the choice of template, coefficients, and variable names. The first test retains the original template-specific unequal examples. Its opposite-label examples match signed literal counts and nineteen source-structure counts.

The second test swaps two variable names throughout the positive right-hand expression to create its unequal partner. This keeps the measured, unlabeled position histograms equal. For example, $2x + y$ and $2y + x$ have the same shape and coefficients but usually differ in value. The test was motivated by a simple histogram classifier’s performance on development validation data, before either test’s neural predictions were read. Matching removes that measured cue, not every possible shortcut. It also changes how unequal examples are made, so the difference between test scores is not a pure causal estimate of reliance on shape.

The finite graphs are checked separately from the source labels. At the structure-matched groups' saved unequal-coordinate counterexamples, the new replay records 1,800 agreeing whole-graph evaluations, 0 mismatches, and 0 inconclusive evaluations. The unequal pair remains distinguishable in 600 of 600 group/precision checks. Unlike equal-coordinate corner tests, these assignments can expose variable-binding mistakes. Historical EML retains its original domain limitation: root agreement permits nonfinite intermediates, while strict finite-intermediate checks remain inconclusive. Its learning results therefore concern compiled syntax paired with source-equivalence labels; they are not validated recognition of equivalent finite-real EML programs.

What the simple shape baseline sees. The motivating baseline is a ridge classifier with penalty 0.001. It counts combinations of node kind, distance from the output, and height above the leaves; both distances are clipped at 63. It omits variable identities and numerical values. Its vocabulary and feature scaling use training data only. Table 4 reports the complete development result for the three representations. Bounded EML's perfect training score shows that this task contains a usable shape cue even without variable or coefficient information. Its lower validation score also demonstrates that fitting that cue is not sufficient for reliable generalization.

Table 4: Shape-only ridge baseline on the 480 training and 120 development-validation pairs. These are single deterministic baseline fits, not neural test results. Neither frozen 600-pair test was used to fit or select this baseline.

Representation	Training accuracy	Validation accuracy
Source	50.00%	50.00%
Historical EML	67.50%	59.17%
Bounded EML	100.00%	65.83%

The variable-swap construction matches these histograms within each opposite-label group. A classifier using only those matched features therefore has exactly 50% accuracy on balanced groups: it must assign the same prediction to both labels. This is a construction check, not an additional empirical discovery of model failure. The neural models receive variable labels and numerical features as well, so their near-chance results require a separate evaluation.

Model selection is part of the result. All nine final training states meet the fitting criterion. The validation-selected states meet it in two of three source-graph runs and none of the six EML runs. Choosing the model with lowest validation loss is legitimate, but it can select a state that has not fitted the training set. We therefore keep those selected models as the primary evaluation and separately test the already-fixed final states. This additional check was specified after the original test results were known. Its endpoint is the last completed training update, chosen without comparing test scores.

Table 5 gives both training accuracy and loss for every selected and final state. All runs reach 3,000 updates. Continued training lowers training loss, while validation loss can increase substantially. The final-state transfer result therefore answers a narrow question: fitting the available training examples does not remove the observed variable-swap failure. It does not establish that more varied training data, a different objective, or another model would fail.

The fitted final models also remain near chance on variable-swap examples. This directly links successful fitting and poor transfer for the same saved model states. It establishes failure on this population across the tested representations, not an EML-specific cause. For the selected models, mean accuracy on the original unequal examples is 69.1% for source graphs, 60.3% for historical EML, and 58.3% for bounded EML. On the variable-swap test, the corresponding values are 49.8%, 50.2%, and 50.0%. The latter observation is a shared failure of the tested models. It does not show that EML uniquely causes a failure to generalize.

Each result includes every seed, including failed fits. We show individual seed points and report their sample standard deviation. Intervals obtained by resampling matched example groups describe sampling variation for these three models, rather than variation across new training runs. Table 5 reports every training state; Appendix C gives individual test scores and template results. Full-precision seed deviations and grouped intervals are in the machine-readable supplementary analyses.

Table 5: All main-study saved states. S: validation-selected; F: final. Seeds 21/22/23 mean 20260921/22/23. Accuracy is a percentage. The joint fitting criterion is accuracy at least 95% and BCE at most 0.2.

Input	Seed	State	Update	Train acc.	Train BCE	Val. acc.	Val. BCE
Source	21	S	1275	100.0	0.00861	94.2	0.217
Source	21	F	3000	100.0	1.47e-05	91.7	0.479
Source	22	S	975	100.0	0.015	96.7	0.0781
Source	22	F	3000	100.0	1.42e-05	94.2	0.39
Source	23	S	375	81.2	0.349	73.3	0.476
Source	23	F	3000	100.0	2.88e-05	82.5	1.15
Historical EML	21	S	375	77.7	0.447	66.7	0.556
Historical EML	21	F	3000	97.3	0.0614	69.2	1.06
Historical EML	22	S	525	84.0	0.33	69.2	0.569
Historical EML	22	F	3000	98.1	0.0387	65.8	1.1
Historical EML	23	S	525	86.2	0.306	70.8	0.539
Historical EML	23	F	3000	96.9	0.0683	69.2	1.01
Bounded EML	21	S	600	82.1	0.397	66.7	0.587
Bounded EML	21	F	3000	98.8	0.0349	67.5	2.02
Bounded EML	22	S	450	69.6	0.531	60.0	0.612
Bounded EML	22	F	3000	99.8	0.00691	68.3	1.97
Bounded EML	23	S	525	76.7	0.445	65.8	0.598
Bounded EML	23	F	3000	99.0	0.0385	71.7	2.57

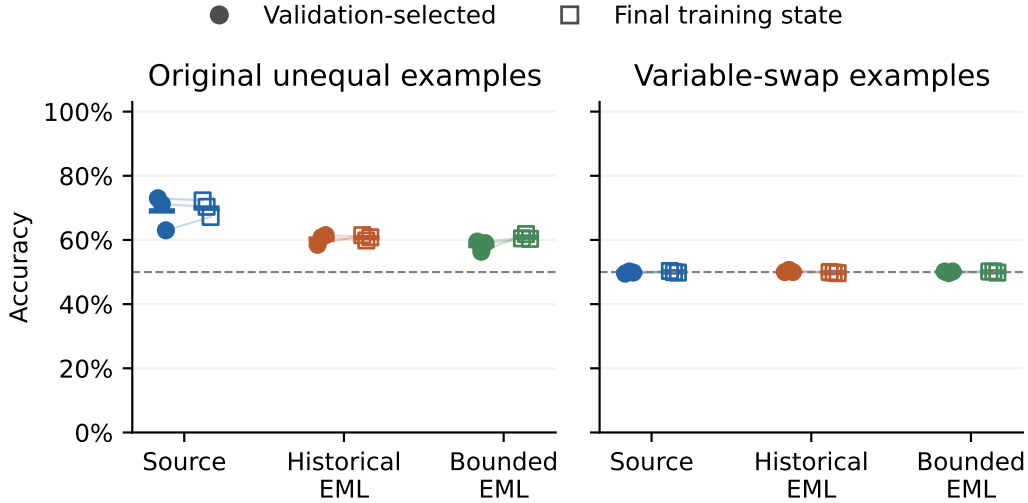


Figure 3: Checkpoint choice and test population. Each point is one of three training seeds; short horizontal marks show means. Lines join selected and final states from the same training run. The dashed line is 50% accuracy. These are seed observations, not group-bootstrap confidence intervals. The final-state comparison is post hoc.

253 **Confidence fails as well as classification.** Variable-swap AUROCs are approximately 0.5 for
254 selected and final models, so changing one classification threshold cannot recover useful overall
255 ordering. Final-state mean BCE is 6.73 for source graphs, 3.47 for historical EML, and 6.22 for
256 bounded EML, compared with 0.693 for balanced constant predictions. Across the three seeds,
257 respectively 41.3%, 30.3%, and 36.9% of all swap examples are wrong with at least 99% predicted
258 confidence. Continued fitting can thus increase confidence in incorrect predictions under this shift.
259 These are measurements on the frozen tests, without threshold fitting or post hoc recalibration.

260 **A small integration with an exact checker.** We replay each test group as two proposed source
261 rewrites, ranked by the saved neural logits. A separate checker expands the restricted polynomial
262 syntax into exact rational coefficient maps and accepts a candidate only when those maps agree. Its

implementation uses Python syntax parsing and rational arithmetic, without consulting the graph compiler, model, stored equivalence label, or numerical witness for acceptance. Its results agree with all 1,200 stored source-pair labels across the two tests.

Table 6: Post hoc two-candidate checker replay and confidence under variable swaps. Calls are mean (sample SD) over three seeds and 300 groups per test; random ordering needs 1.50 calls in expectation. S/F denote selected/final states. “Wrong, $\geq 99\%$ ” is the percentage of all swap examples that are misclassified with at least 99% confidence.

Input	State	Original calls	Swap calls	Swap BCE	Wrong, $\geq 99\%$
Source	S	1.308 (0.066)	1.486 (0.037)	4.79	30.6%
Source	F	1.259 (0.015)	1.498 (0.028)	6.73	41.3%
Historical EML	S	1.388 (0.022)	1.474 (0.031)	1.18	4.0%
Historical EML	F	1.379 (0.022)	1.494 (0.041)	3.47	30.3%
Bounded EML	S	1.391 (0.013)	1.501 (0.025)	1.14	4.4%
Bounded EML	F	1.374 (0.017)	1.496 (0.050)	6.22	36.9%

Each group contains one valid candidate by construction. We count calls until the checker accepts it, averaging both candidate orders for logit differences within 10^{-6} . Table 6 shows useful ordering on the original task but little advantage over random ordering on variable swaps. Group-bootstrap intervals for the latter mean include 1.50 for every model/state combination. This is a post hoc two-candidate integration, not a new generalization population: expected calls equal two minus the existing within-group ranking score. It establishes neither a wall-time speedup nor an end-to-end proof-search improvement. Its practical implication is narrower: model confidence does not justify bypassing the exact checker, and ranking savings must be evaluated on the intended candidate distribution.

Evidence from the larger historical training study. The earlier plain-graph study uses 50,000 training pairs and 5,000 pairs in each validation, IID test, and distribution-stress test. Three-layer width-64 models have 301,956 parameters for both source and historical EML graphs. Mean IID macro-F1 across three seeds is 0.9971 and 0.9802, respectively. The post-training audit of 22,271 endpoints records 18,331 agreements, 2,388 compiled-inconclusive outcomes, 1,551 source-inconclusive outcomes, and one observed mismatch. On 4,492 numerically agreeing IID pairs, macro-F1 is 0.9970 and 0.9793. Filtering this test subset does not repair uncertain training inputs. Strict finite-intermediate checks on the later retained 21,107 historical polynomial endpoints remain inconclusive; the bounded compiler creates different graphs.

This larger experiment establishes that the earlier models learned their original generated task. It cannot serve as a training-size comparison for the current study: the data, architecture, source grammar, and negative examples differ. The current evidence contains one 480-pair main training size. A controlled size sweep and held-out evaluation after training on representative variable-swap negatives remain unmeasured.

6 Implications and limits

Positions enable fitting in these learners, while variable-swap transfer remains poor. A single operator also increases graph size under the tested compiler.

These findings motivate separate checks of compiled behavior, training fit, and variable/coefficient binding in learned rewrite and proof-search guidance. Incorrect equivalence predictions can introduce software defects; independent checking can prevent their acceptance. The replay measures checker calls, while end-to-end benefit remains untested; each proposed transformation still requires verification [Willsey et al., 2021, Polu and Sutskever, 2020].

Controlled learning covers rational polynomials, three seeds, and specific compilers and models; the historical structural corpus is broader. Numerical sampling does not certify the implementation. Neither the position intervention nor these negative results identifies a unique training mechanism or universal limits on EML learning.

References

- Miltiadis Allamanis, Pankajan Chanthirasegaran, Pushmeet Kohli, and Charles Sutton. Learning continuous semantic representations of symbolic expressions. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 80–88, 2017. URL <https://proceedings.mlr.press/v70/allamanis17a.html>.
- Guillaume Lample and François Charton. Deep learning for symbolic mathematics. In *International Conference on Learning Representations*, 2020.
- Andrzej Odrzywółek. All elementary functions from a single binary operator, 2026. arXiv:2603.21852v2, $\text{eml}(x, y) = \exp(x) - \ln(y)$.
- Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving, 2020. arXiv:2009.03393.
- Ladislav Rampasek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a general, powerful, scalable graph transformer. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/5d4834a159f1547b267a05a4e2b7cf5e-Abstract-Conference.html.
- Jorma Rissanen. Modeling by shortest data description. *Automatica*, 1978.
- David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. Analysing mathematical reasoning abilities of neural models. In *International Conference on Learning Representations*, 2019.
- Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, 2021.
- Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *Proceedings of the 35th International Conference on Machine Learning*, pages 5453–5462, 2018. URL <https://proceedings.mlr.press/v80/xu18c.html>.
- Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.

A Task, inputs, and model specification

Task and populations. Each example is a pair of expressions with a binary equivalence label. Every matched group contains one equivalent and one unequal pair with the same left expression. Exact rational-polynomial expansion supplies the labels. The six template families are commutativity, distributivity, regrouping, binomial square, difference of squares, and cubic expansion. Only the first three appear in the 480-pair training population; 120 separate pairs are used for validation. The original two test sets each contain 300 groups, or 600 pairs. Within each set there are 25 groups per template and literal group. Familiar literals are checked against numbers actually present in training, including derived coefficients. An unfamiliar-template cell can also change algebraic structure; these axes do not isolate every possible source of difficulty.

Input encoding. Each node has an operation/variable label, node kind, root flag and root order. Numerical constants have a category, sign, the two values $\log(1 + |p|)$ and $\log(1 + |q|)$ for the exact rational p/q , and a bounded categorical bucket. Previously unseen constants therefore retain numerical features even when their categorical identifier is unfamiliar. A bucket alone is not an injective encoding. All actual input fields are included in the new distinguishability checks.

Each logical parent-to-child edge produces a forward and a reverse message with direction, ordered child slot, and role labels. Repeated references remain repeated messages. Vocabulary capacities are kind 16, label 256, value category 8, value bucket 1024, sign 3, root flag 2, root order 8, direction 2, child slot 8, and role 4. The vocabulary is fitted on training inputs and reused unchanged. Source and historical EML inputs are not silently reconstructed with a new compiler for the main comparison.

Encoder and pair head. Categorical node embeddings and a linear projection of the two numerical features are summed into a width-64 initial state h_v^0 . The positional variant additionally sums two learned 64-entry embeddings: shortest root distance and longest leaf height, clipped at 63. Direction, slot, and role embeddings sum to an edge vector a_{uv} . In layer ℓ ,

$$m_v = \sum_{u \rightarrow v} \text{ReLU}(h_u + W_\ell a_{uv}), \quad u_v = \text{LN}(h_v + \text{MLP}_\ell((1 + \epsilon_\ell)h_v + m_v)).$$

The learned scalar ϵ_ℓ starts at zero. The message MLP has widths 64–64–64 and a ReLU. A second residual block is

$$\tilde{h}_v = \text{LN}(u_v + \text{FFN}_\ell(u_v)), \quad h_v^{\ell+1} = 0.8\tilde{h}_v + 0.2h_v^0,$$

where the feed-forward network has widths 64–128–64 with a ReLU. Layer normalization operates on each node’s feature vector. Dropout is zero. Twelve separate blocks are used in the main study. The graph summary is the mean of the twelve per-layer node means. A shared encoder processes the two expressions; their summaries a, b produce $[a + b, |a - b|, a \odot b]$. A 192–64–1 ReLU network returns a logit. This construction is symmetric under swapping expressions. It does not directly align their nodes.

Optimization and saved models. AdamW uses learning rate 10^{-3} and zero weight decay. Gradients are clipped to norm 1. Batches contain 16 complete two-label groups. Group order is deterministically shuffled each epoch; no group is divided between training and validation. The main fits use at most 3,000 updates, 200 epochs, or 1,800 active seconds, with validation every five epochs. Minimum validation BCE selects the primary model. Final-state inference uses the last already-saved training update and was specified after the original test results were known; it is a post hoc diagnostic. It does not replace the primary selected-model results. Random seeds and continuous training outcomes are listed below.

Metrics and uncertainty. Classification predicts equivalence when $\text{logit} > 0$. Within each matched group, ranking is 1 if the equivalent pair’s logit exceeds the unequal pair’s by more than 10^{-6} , 0 if it is lower by more than that amount, and $1/2$ otherwise. Macro-F1 averages the two class-specific F1 scores. AUROC measures ranking across all positive and negative examples. BCE is the mean binary cross-entropy loss on all pairs. Results include all seeds. The sample standard deviation describes observed seed variation; paired 2,000-draw bootstrap intervals resample complete groups jointly across representations and are conditional on the three fitted seeds. Template/literal aggregates weight pairs equally; each original test cell has the same number of pairs.

378 **New controls and their interpretation.** The binding-development population uses the same
 379 variable-swap negative construction, with 32 balanced familiar-template training pairs and 32 valida-
 380 tion pairs. Exact source strings, polynomial functions, and generator contexts are disjoint from the
 381 earlier populations. The recipe is fixed before fitting. Each tiny fit receives at most 1,500 updates or
 382 900 active seconds and is checked every 50 updates; accuracy $\geq .95$ and BCE $\leq .2$ must hold for
 383 three consecutive checks. The optional global-message extension and a further frozen holdout were
 384 not completed. They contribute no empirical result here. The completed tiny controls assess fitting
 385 only; they do not support a new claim about generalization.

386 B Finite-real construction and numerical checks

387 **Statement and assumptions.** Let an accepted source expression be generated by exact Inte-
 388 ger/Rational leaves, real variables, and ordered Add/Mul. Each variable has a finite closed interval
 389 with rational endpoints compatible with its declared assumptions. The compiler returns a DAG
 390 containing only 1, variables, and $E(a, b) = \exp(a) - \log b$. In exact real arithmetic, for every
 391 assignment in the box, its root equals the source value, every primitive intermediate is finite real, and
 392 every primitive logarithm has a strictly positive operand. This statement depends on the elementary
 393 identities below, exact interval arithmetic, parser correctness, and the implementation of sharing. It is
 394 a compositional mathematical argument with executable checks, not a proof-assistant certificate.

395 **Primitive domain lemmas.** For finite real t , $X(t) = E(t, 1) = \exp(t)$ is strictly positive and finite.
 396 For strictly positive finite t ,

$$L(t) = E(1, X(E(1, t))) = e - \log(\exp(e - \log t)) = \log t.$$

397 Every logarithm in this composition has a positive operand. If $a > 0$ and b is any finite real, then

$$S(a, b) = E(L(a), X(b)) = a - b.$$

398 Its first-operand positivity side condition is essential. With $e = X(1)$ and $e - 1 = S(e, 1)$, define

$$N(t) = S(1, S(e, S(e - 1, t))) = -t.$$

399 The three first operands are the positive constants 1, e , $e - 1$; intermediate subtraction results may
 400 have either sign without violating these conditions. Rational bounds $2 < e < 3$ enclose the helper
 401 constants. Zero is $L(1)$. Positive integers are built by doubling/addition from one; negative integers
 402 use N .

403 **Safe addition.** If the lower bound of a is positive, define $A(a, b) = S(a, N(b))$. For integer $K \geq 0$,

$$T_K(t) = S(1 + K, S(1, t)) = t + K; \quad T_{-K}(t) = S(1, S(1 + K, t)) = t - K.$$

404 These translations require only positive constant first operands. Otherwise, if $l \leq a$ is a rational lower
 405 bound, choose $K = 1 - \lfloor l \rfloor$, giving $a + K \geq 1$, and define

$$A(a, b) = T_{-K}(S(T_K(a), N(b))) = a + b.$$

406 The enclosing output interval is $[a_l + b_l, a_u + b_u]$. Source variables may share dependencies; interval
 407 overapproximation remains safe.

408 **Products and rational constants.** For positive operands, $P(a, b) = X(A(L(a), L(b))) = ab$.
 409 Rational bounds for logarithms follow from $1 - 1/x \leq \log x \leq x - 1$ for $x > 0$. Safe addition
 410 handles a negative or zero logarithm. A strictly negative operand is negated, the product is computed
 411 recursively, and the result is negated. In the remaining zero-containing or sign-crossing cases choose
 412 positive integers K, L with $a + K \geq 1$ and $b + L \geq 1$, and use

$$ab = P(a + K, b + L) - P(a + K, L) - P(K, b + L) + KL.$$

413 Subtraction uses A and N . The resulting interval is the minimum/maximum of the four endpoint
 414 products. For integer $q > 0$, $X(N(L(q))) = 1/q$; multiplication with the signed numerator constructs
 415 a rational without adding numerical leaves.

Induction and sharing. Leaves equal their exact literal values or satisfy their declared variable boxes. Each operation above preserves the source value, carries a sound rational interval, and satisfies every primitive domain condition. Structural induction establishes the statement for every accepted syntax tree. Sharing identical ordered nodes merges only equal subexpressions and preserves repeated child slots. Removing nodes not used by the output affects only unused helpers. Unsupported source syntax is rejected; fixed nonnegative integer powers must be expanded before compilation.

Numerical evidence and its limits. The replay traverses exported primitive nodes child before parent, checking logarithm positivity, intermediate finiteness, and an explicit exponent cap. It does not substitute the macro identities. Exact rational polynomial evaluation supplies source values. All 21,107 source endpoints compile; their largest new graph has 1,463 nodes. Three assignments (both box corners and one source-hashed coordinate assignment) at 80 and 120 digits yield $21,107 \times 3 \times 2 = 126,642$ agreeing whole-graph evaluations. Each graph evaluation checks every traversed primitive node; this total is not a count of individual primitive-node evaluations. Agreement uses scaled tolerance 10^{-20} and exponent cap 10^9 . A prior 512-source run at a lower cap has 3,044 agreements and 28 cap-inconclusives; the latter agree at the same points/precisions in a separately recorded higher-cap replay. The full numerical ledger retains source/graph/certificate identifiers and outcome classes.

Finite real in exact arithmetic does not mean representable in standard hardware floating point. Nested exponentials can have enormous finite magnitudes, and subtraction can be ill-conditioned. The numerical audit is finite sampling, not a universal quantified proof or an independently verified implementation. Conversely, the mathematical domain argument covers the whole declared box subject to its trust boundary, rather than only the sampled assignments. Historical official-v4 graphs keep their original strict-inconclusive outcomes; new finite graphs constitute a different representation and can have different size, distances, and optimization behavior.

Checks sensitive to variable binding. Equal-coordinate box corners are inadequate for detecting swapped variables: $2x + y$ and $2y + x$ agree whenever $x = y$. We therefore evaluate the saved finite graphs at each structure-matched group’s exact unequal-coordinate counterexample. At 80 and 120 decimal digits, we compare the left expression and both right expressions with exact rational values, check every visited primitive’s domain, and check that the unequal pair remains numerically distinct. Outcomes are retained separately as agreement, mismatch, and inconclusive. These checks improve implementation evidence for the tested inputs; they do not turn sampling into a proof.

C Complete current results

Table 5 in the main text retains all training states. The following compact tables retain every test seed. Full precision, template/literal cells, prediction logits, and grouped intervals are also included in the machine-readable supplementary results. A training pass is a description of a particular run, not a precise success probability for future training. The positional model has 457,741 parameters. Each architecture uses the same parameter count across the three representations. All runs use the same RTX 5090. Active times include evaluation and checkpoint work; they describe these runs and are not an isolated inference-speed benchmark. The optional global-message extension and the additional frozen holdout were not completed and supply no reported outcome. The nine completed positional fitting controls above are development results, not new generalization claims. The complete per-group numerical outcomes and source/input hashes are in the supplementary ledger.

D Supporting diagnostics and earlier experiments

Limits of earlier comparisons. The first four-model grid failed to fit even source graphs and introduced an unseen power label. A later comparison expanded powers into multiplication, but concatenating layer outputs changed the comparison-head size with depth. Those results cannot isolate the effect of propagation depth. The archived records retain the original outcomes; the fixed-size and shared-weight controls in the main text address specific limitations of those earlier comparisons.

Table 7: Original unequal examples: all seeds and both model choices, 600 pairs per model. S/F as above.

Input	Seed	State	Acc.%	F1	AUROC	Ranking	BCE
Source	21	S	73.0	0.713	0.808	0.733	2.48
Source	21	F	72.3	0.713	0.802	0.757	3.15
Source	22	S	71.2	0.697	0.772	0.727	3.25
Source	22	F	70.3	0.692	0.782	0.727	3.53
Source	23	S	63.0	0.616	0.689	0.617	0.862
Source	23	F	67.2	0.672	0.744	0.74	2.37
Historical EML	21	S	58.5	0.562	0.617	0.588	0.816
Historical EML	21	F	61.5	0.606	0.64	0.642	2.22
Historical EML	22	S	60.8	0.603	0.654	0.632	0.892
Historical EML	22	F	59.8	0.585	0.667	0.623	2.11
Historical EML	23	S	61.5	0.595	0.625	0.617	1.11
Historical EML	23	F	60.8	0.583	0.632	0.598	2.68
Bounded EML	21	S	59.5	0.594	0.669	0.623	0.77
Bounded EML	21	F	60.5	0.605	0.68	0.608	2.78
Bounded EML	22	S	56.3	0.563	0.632	0.6	0.698
Bounded EML	22	F	61.8	0.618	0.682	0.628	3.18
Bounded EML	23	S	59.0	0.588	0.649	0.603	0.814
Bounded EML	23	F	60.3	0.603	0.67	0.642	3.74

Table 8: Variable-swap examples: all seeds and both model choices, 600 pairs per model. S/F as above.

Input	Seed	State	Acc.%	F1	AUROC	Ranking	BCE
Source	21	S	49.5	0.373	0.5	0.517	6.02
Source	21	F	50.3	0.409	0.5	0.503	7.33
Source	22	S	50.2	0.392	0.503	0.55	6.82
Source	22	F	50.0	0.412	0.499	0.53	7.97
Source	23	S	49.8	0.448	0.5	0.477	1.54
Source	23	F	49.8	0.482	0.499	0.473	4.88
Historical EML	21	S	50.0	0.454	0.503	0.495	0.933
Historical EML	21	F	50.0	0.465	0.501	0.46	3.18
Historical EML	22	S	50.7	0.483	0.502	0.557	1.24
Historical EML	22	F	49.8	0.464	0.502	0.52	3.4
Historical EML	23	S	50.0	0.432	0.5	0.527	1.37
Historical EML	23	F	49.7	0.438	0.504	0.538	3.82
Bounded EML	21	S	50.2	0.495	0.5	0.513	1.2
Bounded EML	21	F	50.2	0.499	0.5	0.548	4.55
Bounded EML	22	S	49.7	0.495	0.498	0.47	0.957
Bounded EML	22	F	50.2	0.5	0.499	0.515	6.3
Bounded EML	23	S	50.2	0.5	0.5	0.513	1.28
Bounded EML	23	F	49.8	0.49	0.5	0.45	7.82

Table 9: Original unequal examples: accuracy in percent, selected / final, averaged equally over three seeds and 100 pairs per template. The first three templates occur in training; the last three do not.

Template	Source S/F	Historical S/F	Bounded S/F
commutativity	96.3 / 95.0	93.7 / 95.0	87.7 / 89.0
distributivity	77.3 / 88.0	54.0 / 58.0	49.3 / 59.0
regrouping	91.0 / 87.0	61.3 / 60.7	66.7 / 70.0
binomial square	50.3 / 50.7	50.7 / 49.3	50.7 / 49.3
difference squares	49.7 / 49.3	50.0 / 50.0	50.3 / 50.3
cubic	49.7 / 49.7	52.0 / 51.3	45.0 / 47.7

Table 10: Variable-swap examples: accuracy in percent, selected / final, averaged equally over three seeds and 100 pairs per template. The first three templates occur in training; the last three do not.

Template	Source S/F	Historical S/F	Bounded S/F
commutativity	50.3 / 50.0	50.0 / 50.0	50.3 / 50.7
distributivity	50.0 / 50.0	50.0 / 50.0	49.7 / 49.3
regrouping	49.7 / 49.3	51.7 / 49.3	50.0 / 50.0
binomial square	50.0 / 50.0	49.7 / 49.7	50.0 / 50.0
difference squares	49.0 / 51.3	50.0 / 50.0	50.0 / 50.0
cubic	50.0 / 49.7	50.0 / 50.0	50.0 / 50.3

Table 11: All tiny variable-binding development fits, 32 training pairs. Seeds 41/42/43 mean 20260941/42/43. The gate requires three consecutive passing checks. Failed runs remain included. Seconds count active work, excluding resource cooldowns.

Model	Input	Seed	Updates	Acc.%	BCE	Gate	Seconds
Position	Source	41	250	100.0	9.98e-05	pass	25.4
Position	Historical EML	41	1500	65.6	0.477	fail	250.7
Position	Bounded EML	41	400	100.0	0.000208	pass	130.6
Position	Source	42	300	100.0	0.000187	pass	34.5
Position	Historical EML	42	1500	65.6	0.477	fail	375.6
Position	Bounded EML	42	700	100.0	9.14e-05	pass	368.0
Position	Source	43	250	100.0	0.000186	pass	48.9
Position	Historical EML	43	1500	65.6	0.485	fail	374.7
Position	Bounded EML	43	350	100.0	0.000352	pass	164.6

Table 12: Full-feature exact color refinement on 300 opposite-label graph pairs per representation. Twelve rounds and normalized color histograms account for mean pooling. Different colors do not guarantee that the fixed parameterization or optimizer learns a separator.

Input	Distinguishable	Indistinguishable	Unresolved
Source	300	0	0
Historical EML	300	0	0
Bounded EML	300	0	0

Table 13: Additional paired structure measurements on the same 2,700 source endpoints. Clipped percentages count nodes with true position greater than 63, weighted equally over all nodes in each representation. Finite-domain construction also changes graph size, depth, and how much position information is compressed.

Input	Mean nodes	Maximum depth	Root clipped%	Leaf clipped%
Source	11	6	0.0	0.0
Historical EML	111	95	0.0	2.0
Bounded EML	323	253	14.0	37.9

The symmetric six-choice search gives five source-graph passes and zero historical-EML passes; the subsequent 16 position controls all pass. The first failed grid and corrected concatenated-readout grid remain separate in the archived result records. For the initial historical-EML model with a fixed-width summary, the FP32 median matched distance is 5.82e-07; reversing message order for the same graph gives median distance 5.32e-07. This is one measured numerical perturbation, not a complete model of numerical error.

Table 14: Higher-precision replay of identical fixed-readout weights. Relative separation divides matched distance by the mean norm of the two summaries. This removes a common scalar scale factor; it does not identify a unique training mechanism.

Input	State	Absolute separation	Relative separation
Source	initial	0.0249	0.00479
Source	trained	3.47	0.799
Historical EML	initial	2.25e-10	4.12e-11
Historical EML	trained	2.91e-15	5.31e-16

471

Ordering can survive a failed threshold. The original trained models give a distinct transfer observation. Table 15 shows every template, rather than singling out favorable commutativity results. The population differs from the current tests. Useful within-group ordering is not a sound equivalence certificate.

Table 15: Historical trained-model transfer: 100 matched groups per template, mean and sample standard deviation over three original seeds.

Matched-pair ranking			
Template	AST	EML	Prefix
Commutativity	0.863 (0.024)	0.470 (0.052)	0.017 (0.006)
Distributivity	0.522 (0.032)	0.500 (0.000)	0.458 (0.028)
Regrouping	0.515 (0.018)	0.500 (0.000)	0.532 (0.170)
Binomial square	0.518 (0.033)	0.503 (0.025)	0.467 (0.055)
Difference of squares	0.480 (0.082)	0.427 (0.030)	0.560 (0.056)
Cubic	0.458 (0.025)	0.500 (0.000)	0.550 (0.018)
All templates	0.559 (0.029)	0.483 (0.005)	0.431 (0.032)

Accuracy at logit > 0			
Template	AST	EML	Prefix
Commutativity	0.500 (0.000)	0.500 (0.000)	0.500 (0.000)
Distributivity	0.500 (0.000)	0.500 (0.000)	0.502 (0.003)
Regrouping	0.497 (0.006)	0.500 (0.000)	0.505 (0.020)
Binomial square	0.500 (0.000)	0.500 (0.000)	0.500 (0.000)
Difference of squares	0.500 (0.000)	0.500 (0.000)	0.500 (0.000)
Cubic	0.500 (0.000)	0.500 (0.000)	0.500 (0.000)
All templates	0.499 (0.001)	0.500 (0.000)	0.501 (0.004)

475

476 E Artifact access and reproducibility

The supplementary ZIP includes development/test graphs and source pairs, all main-comparison selected and final model tensors, saved prediction logits, full result cells, the finite-compiler audit ledgers, and analysis/source code. The file `index.json` maps models to their inputs and predictions; `manifest.json` binds exported files. The 250,000-expression corpus and historical learning weights are not bundled; only their summaries are included. Tiny binding-control weights and optimizer states are also excluded to limit size; their inputs and fitting records remain included. The optional global-message extension and additional holdout inference were not completed; no outcomes are claimed for them. These availability limits apply to the supplementary package, independently of

485 retained author archives. The package's README.md gives exact verification and replay commands.
486 The package provides a script that checks every listed file against its manifest and a separate command
487 that rebuilds the current results tables and plots from saved predictions. Running the same checker
488 again is reproducibility evidence, not an independent proof of its correctness. Historical source/model
489 archives that are not included are explicitly identified; the paper does not imply that every author-side
490 record is accessible in the submitted supplement.

491 **Checker replay and confidence analysis.** The new supplementary script verifies exact polynomial
492 identities with rational coefficient maps and then replays the saved candidate order. It retains all 36
493 model/state/test combinations and all group-level call counts. Nine exact-arithmetic fixtures and six
494 unsupported-input rejection fixtures pass; source-label disagreements and failed candidate groups
495 are zero. Parser and arithmetic implementation correctness remain a trust boundary. The grouped
496 intervals use 2,000 matched-group bootstrap draws conditional on the same three training seeds. The
497 replay does not test newly generated candidates, unequal checker costs, or training on variable-swap
498 negatives. The supplement also contains the complete historical generation configurations, recorded
499 size/depth distributions, source/compiler identifiers, and regeneration commands. These improve
500 specification without claiming a fresh 250,000-expression regeneration.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Introduction state the three contributions and restrict the learning findings to the tested compilers, populations, and models. Section 6 distinguishes these findings from universal learnability or verification claims.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 and Appendices B–E discuss grammar restrictions, finite numerical evidence, small training populations, architecture and position effects, post hoc analyses, and unavailable historical artifacts.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Section 2 sketches the bounded construction; Appendix B states its grammar, interval and exact-arithmetic assumptions, gives the arithmetic identities, and explains induction and sharing. This mathematical argument does not certify the implementation or guarantee floating-point stability.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [No]

Justification: The current controlled experiments include detailed methods, data, saved models, predictions, and replay code (Appendices A, C, and E). The complete historical corpus and weights are excluded, and its recorded generation configuration has not been freshly replayed in a fully reconstructed original environment.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The anonymized supplementary ZIP provides code, current data, model tensors, and exact replay commands, but not the entire historical corpus or historical model weights. Appendix E lists these exclusions, so access is incomplete for the full set of reported experiments.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Sections 2 and 5 and Appendix A specify the populations, splits, features, architecture, optimizer, budgets, selection rules, and seeds. Selected and final checkpoints and development-only controls are explicitly distinguished.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Section 5 and Appendix A define sample standard deviations across training seeds and 2,000-draw matched-group bootstrap intervals conditional on those seeds. All individual seeds are retained; the intervals do not claim to capture variation across new training runs.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: Appendices A and C identify the RTX 5090, run budgets, and active run times, and the supplement records software versions. Complete per-experiment peak memory and project-wide compute, including all historical and preliminary work, are not available in one complete accounting.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This synthetic-expression study involves no human participants or personal data; failed experiments and consequential limitations remain disclosed. Section 6 addresses unsafe rewrite acceptance, Section 2 discloses LLM assistance, and the release is anonymized.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 6 discusses the potential benefit of independent checking and the risk that incorrect equivalence predictions introduce software defects. Practical end-to-end gains and deployment safety are not claimed.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released assets are synthetic symbolic expressions and restricted graph models, not general-purpose generative models or scraped personal data. Section 6 nevertheless requires independent verification before accepting proposed transformations.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: The manuscript credits the original operator and prior learning methods, and the package includes an MIT license for the supplied software. A complete asset-by-asset inventory of third-party licenses and terms is not supplied; therefore we do not claim complete license documentation.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: Appendix E and the supplementary README, index, manifests, environment record, and generation configurations document the new code, synthetic inputs, saved models, and replay procedures. Intended scope and excluded artifacts are explicit.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The study uses generated mathematical expressions and computational experiments; it involves no crowdsourcing or human-subject experiments.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human participants, participant data, or participant interventions are involved, so human-subject ethics approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: The LLM-assistance paragraph in Section 2 describes coding, debugging, experimental design and execution, analysis and interpretation assistance, manuscript refinement, and author-side AI reviews. It distinguishes this assistance from executable ground-truth checks and from the evaluated models.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.